Que son?

Hemos sido testigos de como existen distintos tipos de error que pueden pasar durante la ejecución del código.

Es decir, mientras el código está corriendo. **No antes**.

A este tipo de errores *no sintácticos* le llamamos `Exceptions`.

Es decir que son errores que suceden con código escrito con sintaxis valida.

Esto **no** es una excepción ya que el código nunca llega a ejecutarse:

Copiar

```
1  def main():
2      print('Ejecutando...')
3      &^&*&^*&^
4
5
6  if __name__ == '__main__':
7    main()
```

File "c:\Users\alekc\repos\d0ad\Secciones\Excepciones\video.py", line 3
(&&^)
^
SyntaxError: invalid syntax

Por otro lado, digamos que intentamos accesar a un índice que no existe en una lista:

```python
def main():
  my_4_elements = ["One", "Two", "Three", "Four"]

  print(my_4_elements[3])
  print(my_4_elements[4])


if __name__ == '__main__':
  main()
```

Copiar



```
Four
Traceback (most recent call last):
File "c:\Users\alekc\repos\d0ad\Secciones\Excepciones\index_out_of_range.py", line 4, in <module>
print(my_4_elements[4])
~~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

La mayoría de excepciones tienen un nombre.

En el caso anterior, tenemos una excepción de tipo `IndexError`.

Otro ejemplo es cuando intentamos convertir un string **no numeral** a un int:

```python
def main():
  my_first_string = "2"
  my_second_string = "Hello"

  my_first_int = int(my_first_string)
  print(my_first_int + 2)

  my_second_int = int(my_second_string)
  print(my_second_int + 2)


if __name__ == '__main__':
  main()
```

Copiar

```
4
Traceback (most recent call last):
File "c:\Users\alekc\repos\d0ad\Secciones\Excepciones\int.py", line 7, in <module>
my_second_int = int(my_second_string)
^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'Hello'
```

En este caso, tenemos una excepción de tipo `ValueError`.

Cómo manejarlas

Todos los códigos tienen `happy paths` (*caminos felices*) en los que el usuario usa el programa exactamente como lo planeamos.

Sin embargo, también existen los `unhappy paths` (*caminos no felices*) en los que el usuario hace algo no planeado que causa una excepción.

Por ejemplo, escribir *"Hola"* cuando le pedimos un número.

Como desarrolladores tenemos que tomar en cuenta tanto los `happy paths` como los `unhappy paths` y manejar ambos para que el programa no se caiga.

Esto lo podemos lograr gracias a la sintaxis de `try` y `except`.

En la mayoría de los lenguajes es `try` y `catch`, pero el resto de la sintaxis suele ser igual.

Esta nos permite agregar instrucciones que podrían arrojar un `Exception` y definir instrucciones para qué hacer en caso de que eso suceda.

Veamos el mismo ejemplo con un `try - except` implementado:

Copiar

```python
def main():
  my_second_string = 'Hello'

  try:
    my_second_int = int(my_second_string)
  except ValueError:
    print('Hubo un error al convertir este string a numero!')
  print(my_second_int + 2)


if __name__ == '__main__':
  main()
```

```
Hubo un error al convertir este string a numero!
```

Tambien podemos darle un identificador al error para mostrarlo como parte de nuestro mensaje:

Copiar

```python
def main():
  my_second_string = 'Hello'

  try:
    my_second_int = int(my_second_string)
  except ValueError as error:
    print(f'Hubo un error al convertir este string a numero: {error}')
  print(my_second_int + 2)


if __name__ == '__main__':
  main()
```

Hubo un error al convertir este string a numero: invalid literal for int() with base 10: 'Hello'

Tambien podemos declarar varios `excepts` en un solo `try` para manejar distintos tipos de excepciones distintos dentro del mismo bloque de código:

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 4

  try:
    list_element_to_convert = my_list[inc
    element_to_int = int(list_element_to_
    print(element_to_int)
  except IndexError as error:
    print(f'El indice a usar no existe er
  except ValueError as error:
    print(f'El elemento de la lista no es

if __name__ == '__main__':
  main()
```

Copiar

El indice a usar no existe en la lista. Error: list index out of range

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 1

  try:
    list_element_to_convert = my_list[inc
    element_to_int = int(list_element_to_
    print(element_to_int)
  except IndexError as error:
    print(f'El indice a usar no existe er
  except ValueError as error:
    print(f'El elemento de la lista no es

if __name__ == '__main__':
  main()
```

Copiar

El elemento de la lista no es un numero valido. Error: invalid literal for int() with base 10: 'Hello'

Si no sé el tipo especifico de la excepción que quiero agarrar, tambien puedo hacer un except del tipo `Exception`. Este atrapará cualquier tipo de excepcion que suceda.

> Sin embargo, lo mejor es siempre estar consciente de qué tipos de excepción puedan pasar y atraparlos específicamente.

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 4

  try:
    list_element_to_convert = my_list[inc
    element_to_int = int(list_element_to_
    print(element_to_int)
  except Exception as error:
    print(f'Ha ocurrido un error: {error}
```

Copiar

Ha ocurrido un error: list index out of range

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 1

  try:
    list_element_to_convert = my_list[inc
    element_to_int = int(list_element_to_
    print(element_to_int)
  except Exception as error:
    print(f'Ha ocurrido un error: {error}
```

Copiar

Ha ocurrido un error: invalid literal for int() with base 10: 'Hello'

Buenas prácticas

Siempre debemos hacer `try` de instrucciones que sabemos pueden ocasionar una excepción.

Si un error puede pasar, va a pasar!

Siempre hay que estar a la defensiva.

Lo mejor es no declarar variables dentro de un `try` ya que si este falla, la variable no existirá y podrá ocasionar errores en futuras instrucciones.

Copiar

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 4

  try:
    list_element_to_convert = my_list[index_to_use]
    element_to_int = int(list_element_to_convert)
  except Exception as error:
    print(f'Ha ocurrido un error: {error}')

  print(element_to_int)


if __name__ == '__main__':
  main()
```

❌

```
Ha ocurrido un error: list index out of range
Traceback (most recent call last):
File "c:\Users\alekc\repos\d0ad\Secciones\Excepciones\variables_in_try.py", line 10, in <module>
print(element_to_int)
^^^^^^^^^^^^^^^^
NameError: name 'element_to_int' is not defined
```

Es mejor declararlas antes del try con valores placeholder.

Copiar

```python
def main():
  my_list = [
    '2',
    'Hello'
  ]
  index_to_use = 4

  list_element_to_convert = '0'
  element_to_int = 0
  try:
    list_element_to_convert = my_list[index_to_use]
    element_to_int = int(list_element_to_convert)
    print(list_element_to_convert)
  except Exception as error:
    print(f'Ha ocurrido un error: {error}')

  print(list_element_to_convert)


if __name__ == '__main__':
  main()
```

Ha ocurrido un error: list index out of range
0

Generalmente es buena practica el envolver todo el programa en un `try - except general`, y dentro de cada función se usar `try - excepts especificos` para cada tipo de error.

Copiar

```python
def function_1():
  try:
    some_logic_with_value_errors()
  except ValueError as ex:
    print(f'An error ocurred in function_1')


def function_2():
  try:
    some_logic_with_index_errors()
  except IndexError as ex:
    print(f'An error ocurred in function_2')


def main():
  try:
    function_1():
    function_2():

  except Exception as ex:
    print(f'An unexpected error ocurred: {ex}')


if __name__ == '__main__':
  main()
```

Nunca debemos usar un `except` para un happy path.

Está muy mal visto ya que está en contra del uso de la sintaxis.

Una vez despidieron a un compañero por esto.

Por ejemplo, usarlo para validar que un texto no sea un número:

Incorrecto

```python
name = input("Ingrese su nombre: ")
try:
    int(name)
    # unhappy path
    print("Su nombre no puede ser un numero!")
except Exception as error:
    # happy path
    edad = input(f"Gracias {name}! Ahora ingrese su edad: ")
    empleo = input(f"Gracias {name}! Ahora ingrese su empleo: ")
```

Copiar

Correcto

```python
try:
    name = input("Ingrese su nombre: ")
    if name.isdigit():
      raise ValueError()
except Exception as error:
    # unhappy path
    print("Su nombre no puede ser un numero!")

# happy path
edad = input(f"Gracias {name}! Ahora ingrese su edad: ")
empleo = input(f"Gracias {name}! Ahora ingrese su empleo: ")
```

Copiar

Cómo arrojar Exceptions

Los exceptions no son necesariamente malos.

A veces hay casos en los que podemos usarlos para tener un mejor control de lo que no debería ocurrir en algún flujo de lógica.

Podemos arrojar exceptions manualmente usando la palabra `raise` seguida del tipo de exception que queramos arrojar.

Por ejemplo, en este código le pedimos la edad al usuario.

Tenemos un try - except en caso de que el usuario no ingrese un número.

Pero también hacemos raise de un `ValueError` en caso de que el número no esté dentro del rango de 1 a 100.

Luego el except de ese `ValueError` le hace raise a la misma excepción para que lo atrape el except de afuera y el programa se cierre.

Copiar

```python
def ask_for_user_information():
  try:
    age = int(input('Ingrese su edad'))
    if age < 1 or age > 100:
      raise ValueError()

  except ValueError as ex:
    print("Ingrese una edad valida!")
    raise ex


def main():
  try:
    ask_for_user_information()
    # create_order()

  except Exception as ex:
    exit()


if __name__ == '__main__':
  main()
```

Tambien podemos hacer exceptions con distintos mensajes:

Copiar

```python
def check_if_number_is_100(number):
  if number < 100:
    raise ValueError('El numero es muy bajo')
  elif number > 100:
    raise ValueError('El numero es muy alto')

  return True

def main():
  number = input('Ingrese un numero')
  try:
    number_int = int(number)
    check_if_number_is_100(number_int)
  except ValueError as ex:
    print(ex)


if __name__ == '__main__':
  main()
```

Ingresando un 500

El numero es muy alto

Ingresando un 50

El numero es muy bajo